

บทที่ 2

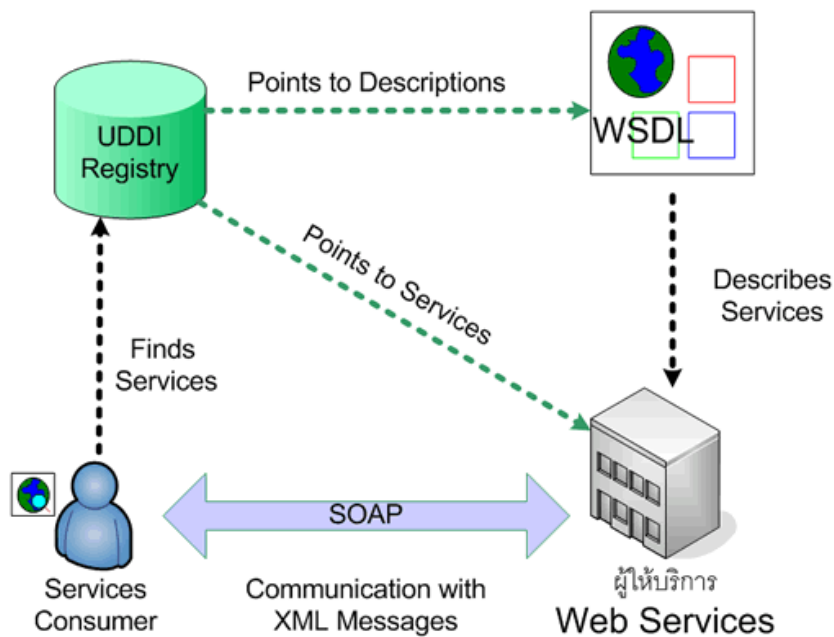
ทฤษฎีพื้นฐานที่ใช้ในโครงงาน

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย และพื้นฐานของระบบเว็บเซอร์วิส ซึ่งเนื้อหาในบทนี้จะภาษากลางที่ใช้ในการติดต่อสื่อสารระหว่างแอปพลิเคชันที่มีความแตกต่างกันหรือที่รู้จักกันในนามภาษา “XML” และเทคโนโลยีที่เกี่ยวข้อง ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษา และพัฒนาระบบเว็บเซอร์วิส

2.1 เว็บเซอร์วิสเทคโนโลยี

โลกของอินเทอร์เน็ตได้พัฒนาไปอย่างรวดเร็วมาก มีการเปิดให้บริการต่างๆ บนอินเทอร์เน็ต ที่เรียกว่าเว็บเซอร์วิส และมีการเชื่อมโยงเว็บเซอร์วิสต่างๆ เข้าด้วยกันเพื่อให้บริการผู้ใช้ได้อย่างรวดเร็วและมีประสิทธิภาพมากขึ้น แต่อุปสรรคที่สำคัญอย่างหนึ่งก็คือ ภาษา HTML นั้นถูกพัฒนาขึ้นมาเพื่อใช้แสดงผลข้อมูลของเว็บไซต์ ให้ผู้ใช้คอมพิวเตอร์ได้ดูผ่านเบราเซอร์ (Browser) เป็นสำคัญ ไม่สะดวกที่จะนำมาใช้แลกเปลี่ยนหรือประมวลผลข้อมูลระหว่างโปรแกรมต่างๆ บนอุปกรณ์คอมพิวเตอร์ด้วยกัน บริษัทและองค์กรต่าง ๆ ที่เกี่ยวข้อง จึงได้ร่วมมือกันจัดทำมาตรฐานภาษาสากลขึ้นมาใหม่ที่เรียกว่า XML (Extensible Markup Language) เพื่อให้สามารถแลกเปลี่ยนข้อมูลได้ทั้งผู้ใช้กับโปรแกรม หรือระหว่างโปรแกรมกับโปรแกรมด้วยกัน (A2A, Application to Application Interoperability) โดยภาษา XML นี้เป็นมาตรฐานเปิด สามารถใช้งานได้ฟรี และไม่ขึ้นกับแพลตฟอร์มใด ๆ สามารถใช้กับระบบปฏิบัติการ (Operating System) ต่างๆ ไม่ว่าจะเป็นวินโดวส์ (Windows) หรือลินุกซ์ (Linux)

ในการเปิดให้บริการเว็บเซอร์วิสนั้น นอกจากจะต้องใช้ภาษาสากลที่เป็นมาตรฐานคือ XML แล้ว ยังเกี่ยวข้องกับเทคโนโลยีอีก 3 ประการคือ



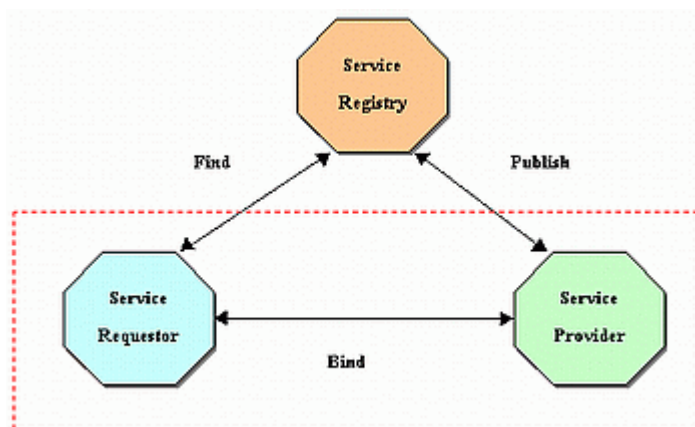
รูปที่ 2.1 ส่วนประกอบหลักของเทคนิคในเว็บเซอร์วิส
เว็บเซอร์วิส = SOAP+WSDL+UDDI

- 1.) Simple Object Access Protocol (SOAP) เป็นโปรโตคอล (Protocol) ที่ใช้ในการรับส่งข้อมูลซึ่งใช้งานบนภาษา XML มีขนาดเล็กและง่าย
- 2.) Web Service Description Language (WSDL) เป็นข้อมูลบนภาษา XML ที่ ผู้ให้บริการ Web-Service จัดทำขึ้นเพื่อเป็นข้อมูลสำหรับการเรียกใช้งานจากเว็บเซอร์วิสนั้นๆ
- 3.) Universal Description, Discovery and Integration (UDDI) เป็นเหมือนเสิร์ชเอนจิน (Search Engine) ตัวหนึ่งที่ผู้ให้บริการเว็บเซอร์วิสสามารถไปขึ้นทะเบียนการให้บริการของตน และผู้เรียกใช้สามารถไปสืบค้นและ/หรือเชื่อมโยงเข้ากับเว็บเซอร์วิสของตนสนใจสามารถดูรายละเอียดเพิ่มเติมได้ที่ www.uddi.org

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่ในเครื่องอื่นโดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ในปัจจุบันที่ใช้มีการสื่อสารระหว่างอ็อบเจ็กต์ (Object) ในระยะไกล (Remote Procedure Calls : RPC) เครือข่ายที่ปัจจุบันใช้งานแพร่หลาย และสะดวกมากที่สุดได้แก่เครือข่ายอินเทอร์เน็ต (Internet) ซึ่งให้การสื่อสารผ่านโปรโตคอลเอชทีทีพี (HTTP) เป็นส่วนใหญ่ เทคนิคระยะไกล (RPC) ของเทคโนโลยีที่กล่าวข้างต้นนั้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วนของไฟร์วอลล์ (Firewalls) และพร็อกซี

เซิร์ฟเวอร์ (Proxy Server) ด้วยเนื่องจากโดยปกติ เซิร์ฟเวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอล HTTP ออกไป เพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการให้บริการเว็บเซอร์วิส ก็คือให้ทำงานอยู่บน โปรโตคอลเอชทีทีพี ซึ่งโพรพนอกจากจะทำงานบนโปรโตคอลเอชทีทีพีแล้วยังเป็นมาตรฐานเปิดที่จะทำให้สามารถติดต่อกับสื่อสารกับเครื่องคอมพิวเตอร์ที่มีความแตกต่างกันทั้งระบบปฏิบัติการ, เทคโนโลยี รวมไปถึงภาษาที่ใช้ในการพัฒนาด้วยก็ได้



รูปที่2.2 รูปโครงสร้างการให้บริการ

การติดต่อกันระหว่างผู้ขอใช้บริการ (Service Requestor) และผู้ให้บริการ (Service Provider) จะเป็นการคุยกันระหว่างแอปพลิเคชันกับแอปพลิเคชัน (Application-to-Application) ซึ่งวิธีการพูดคุยหรือการเชื่อมต่อกันนั้นจะใช้โพรพเป็นตัวกลาง

2.2. การพัฒนาของเอ็กซ์เอ็มแอล (XML) โปรโตคอล

โพรพมีความสัมพันธ์กับ XML โปรโตคอลเป็นอย่างมากเพราะโพรพมีโครงสร้างเป็น XML ซึ่งเทคโนโลยีของ XML เองนั้นก็ได้มีการพัฒนาเพิ่มขึ้นเป็นอย่างมาก ในยุคแรก XML จะถูกใช้งานในลักษณะของข้อมูลตามมาตรฐาน XML 1.0 เช่น WDDX (Web Distributed Data Exchange), XML-RPC

ต่อมาในยุคที่สองได้มีการเพิ่มความสามารถของ XML ในเรื่องของ XML Namespaces และ Schema ซึ่งโพรพจัดว่าอยู่ในยุคที่สองของ XML โปรโตคอล ในยุคที่สองของ XML นั้นได้กำจัดจุดอ่อนของยุคแรกในสองข้อหลักคือ

2.2.1. ในยุคแรกจะมีจุดอ่อนในด้านการเปลี่ยนแปลงโครงสร้างเอกสาร เช่นใน XML-RPC หรือ WDDX ถ้าต้องการเพิ่มให้โปรโตคอลรองรับข้อมูลใหม่ เช่น ข้อมูลไบนารี ทั้ง XML-RPC และ WDDX จะต้องทำการอัปเดตมาตรฐานของตนใหม่ ทำให้ผู้ที่เขียนโปรแกรมในภาษาต่างๆ จะต้องมีการเปลี่ยนแปลงตามไปด้วยซึ่งทำให้การเปลี่ยนเวอร์ชัน (Version) ของมาตรฐานไม่คล่องตัว แต่ในยุคที่สองของ XML โปรโตคอล จะใช้เนมสเปซ (Namespaces) เข้ามาช่วยในการแก้ปัญหานี้

2.2.2. เรื่องที่สองคือการทำงานกับข้อมูลที่มีหลายชนิด (Data type) เนื่องจากในยุคแรกจะใช้ดีทีดี Document Type Definition (DTD) อธิบายโครงสร้างของเอกสาร XML ซึ่งดีทีดี จะไม่มีส่วนของการอธิบายความหมายของชนิดข้อมูล แต่ในยุคที่สองจะใช้ XML Schema แทนซึ่งจะมีการอธิบายทั้งรูปแบบโครงสร้างของลำดับ (syntax) และข้อมูลของชนิดของข้อมูลด้วย

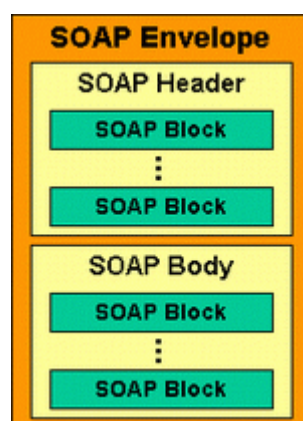
2.3. โครงสร้างของโซฟ (SOAP)

เอกสารโซฟนั้นมีโครงสร้างในรูปแบบ XML ซึ่งเราสามารถแบ่งเป็นส่วนๆของเอกสารได้เป็น 3 ส่วนหลักดังนี้คือ

2.3.1. SOAP envelop เนื้อหาสาระ (Content) ของเอกสารทั้งหมด

2.3.2. SOAP header ส่วนเพิ่มเติมของเอกสาร โซฟซึ่งจะมีก็ได้ หรือ ไม่มีก็ได้

2.3.3. SOAP body ส่วนที่ใช้ในการเรียกใช้งานเซอร์วิส และผลลัพธ์ที่ได้จากเซอร์วิส



รูปที่2.3 รูปโครงสร้างการให้บริการ

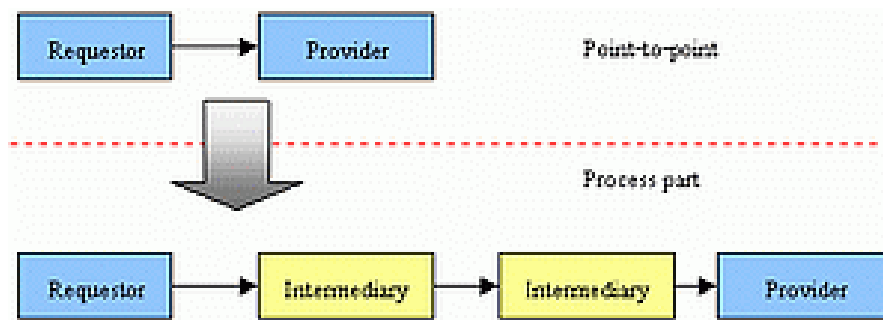
ตัวอย่างเอกสาร SOAP อย่างง่ายของการสอบถามราคาหน่วยความจำ 256 MB ดังตัวอย่างข้างล่างนี้

```
<soap:Envelope>
  <soap:Body>
    <GetPrice>
      <Item>RAM256AQ222</Item>
    </GetPrice>
  </soap:Body>
</soap:Envelope>
```

จะเห็นว่า SOAP Envelope ก็เป็นเอกสารที่อยู่ในรูปแบบ XML ทั่วไป แต่สิ่งที่ทำให้ SOAP มีความสามารถมากขึ้นก็คือ SOAP ได้มีการกำหนดโครงสร้างของเอกสารเป็นส่วนๆ โดยอธิบายว่าส่วนใดมีหน้าที่อะไร มีข้อมูลอะไรอยู่ในส่วนนั้น และใครคือผู้ที่ต้องสนใจในส่วนนั้น ๆ และกล่าวถึงว่าส่วนใดจำเป็นที่ต้องมีในเอกสาร และส่วนใดเป็นส่วนที่เพิ่มเติมอาจจะมีหรือไม่มีส่วนนั้นก็ได้ขึ้นอยู่กับความจำเป็นในการใช้งาน ทำให้การพัฒนาโปรแกรมมีทิศทางที่ค่อนข้างชัดเจน แต่สามารถเพิ่มขยายได้ในอนาคตอีกด้วย

2.4. ตัวกลางในการส่งเอกสาร SOAP (SOAP Intermediaries)

จากตัวอย่างเอกสาร SOAP นั้นจะเห็นได้ว่าในส่วนของ <Header> Element จะเป็นส่วนที่เราจะขยายสารสนเทศ (information) ใหม่ๆ เข้าไปใน SOAP ได้ซึ่งตัวอย่างเรามีการอธิบายให้ผู้รับเอกสารตัวอย่างนี้ให้ทราบว่าในการสอบถามราคานี้ให้ใช้เงินในสกุลเงินบาท เป็นต้น ซึ่งเราจะเรียกการขยายในส่วนนี้ว่าเป็นการขยายในแนวดิ่ง (Vertical extensibility) แต่ถ้าเราจะมาดูในด้านการขยายในแนวนอน (Horizontal extensibility) ของ SOAP นั้นก็คือทำอะไรที่จะให้มีการส่งเอกสาร SOAP ไปยังปลายทางโดยเส้นทางของการเดินทางของเอกสารอาจจะไม่เป็นลักษณะแบบจุดต่อจุด (Point-to-point) แต่อาจจะเป็นการส่งเอกสารตามเส้นทางโดยเอกสารยังคงเป็นเอกสารตัวเดิมดังรูป



รูปที่ 2.4 การทำงานที่มีการส่งเอกสาร โซฟแบบจุดต่อจุด และแบบส่งตามเส้นทาง

ตัวกลางในการส่งเอกสารโซฟจากต้นทาง ในที่นี้คือ “Requestor” ไปยัง “Provider” เพื่อสร้างกระบวนการบางอย่างกับเอกสารซึ่งในการสร้างเส้นทางในการส่งแบบนี้อาจมีจุดประสงค์ดังต่อไปนี้คือ

2.4.1. ทำหน้าที่เป็น Crossing trust domains เนื่องจากการทำงานของ Web Services เป็นการเรียกใช้งานแบบระยะไกล หรือมีการทำงานแบบระบบกระจาย (Distributed System) ดังนั้นวิธีการในด้านความปลอดภัยจึงเป็นส่วนหนึ่งที่ต้องคำนึงถึง ซึ่งวิธีการหนึ่งก็คือให้มีการส่งเอกสารมายังจุดหนึ่งที่มีความน่าเชื่อถือได้ก่อน แล้วส่งต่อไปยังปลายทางอีกที่หนึ่ง ซึ่งหลักการใช้ Trust Domains เราอาจจะดูตัวอย่างง่ายๆ เช่น ในองค์กรเราอาจจะมีเครือข่ายอินทราเน็ต (Intranet) อยู่ภายในองค์กร เครื่องคอมพิวเตอร์แต่ละเครื่องอาจจะสามารถติดต่อไปยังเซิร์ฟเวอร์ (Server) ได้โดยตรง แต่ถ้าต้องการออกไปยังอินเทอร์เน็ต หรือ เอ็กซ์ทราเน็ต (Extranet) ที่อยู่นอกองค์กรจะต้องผ่าน Trust Domains ก่อน ในที่นี้อาจจะเป็นไฟลล์วอลล์ หรือ Virtual Private Network (VPN) gateways เพื่อความปลอดภัย

2.4.2. ทำหน้าที่เป็นตัวขยายการทำงานแบบระบบกระจาย (Distributed System) ถ้าเรามองดูเฉพาะแง่ของการขอใช้บริการจาก Requestor ไปยังผู้ให้บริการ Provider โดยผู้ให้บริการจะมีการกระบวนการทำงานบางอย่างภายในและตอบกลับไปยังผู้ขอใช้ ซึ่งการทำงานแบบนี้ก็อยู่บนพื้นฐานของ Request and Respond นั้นเอง แต่ถ้าระบบของเรามีการขยายใหญ่ขึ้นสิ่งที่เราอาจจะต้องดูเพิ่มขึ้นก็คือความสามารถในการรองรับงานของผู้ให้บริการด้วย ดังนั้นก่อนที่จะมีการส่งเอกสารโซฟไปยังเซิร์ฟเวอร์ ของผู้ให้บริการนั้นอาจจะมีการผ่าน Intermediary Server ก่อนเพื่อทำหน้าที่เป็นบัฟเฟอร์ (Buffer) รับเอกสารโซฟมาพักไว้และส่งเอกสารให้ผู้ให้บริการ (Provider) อีกทอดหนึ่ง การทำงานเช่นนี้จะช่วยให้เซิร์ฟเวอร์ของผู้ให้บริการไม่รับภาระงานมากเกินไปในช่วงเวลาใดเวลาหนึ่งจนอาจทำให้เกิดความเสียหายได้และ Intermediary อาจจะมีการรวบรวมเอกสารแล้วส่งมาให้ผู้ให้บริการเป็นลักษณะ Batch เพื่อเพิ่มประสิทธิภาพ

การใช้งาน Intermediary ลักษณะนี้ผู้ขอใช้บริการอาจจะไม่เห็น Intermediary แต่จะมองเห็นว่าติดต่อกับผู้ให้บริการโดยตรง

2.4.3. การใช้งาน Intermediary ในลักษณะ Value-added ให้กับระบบ เช่น ใช้ Intermediary ในการเข้ารหัสข้อมูล (Encrypts) และเซ็นลายเซ็นอิเล็กทรอนิกส์ (Digital signs) ในการส่งและรับ หรืออีกตัวอย่างหนึ่งเช่นต้องการเก็บข้อมูลเพิ่มนำไปวิเคราะห์การทำงานของระบบโดยการรับ-ส่งเอกสารจะผ่าน Intermediary แต่ไม่ได้ทำการประมวลผลเอกสารแต่จะเก็บของข้อมูลบางอย่างไว้เท่านั้นเช่นนำไปวิเคราะห์ค่า QoS (Quality of Service) หรือ Bottleneck ของระบบ เป็นต้น

2.5. Intermediaries ใน SOAP

เอกสารSOAPจะถูกส่งจากผู้ขอใช้บริการผ่าน Intermediaries เป็นไปจนถึงปลายทางคือผู้ให้บริการ ดังนั้นเอกสารSOAPก็ถูกเปิดเผยทั้งหมดกับ Intermediaries ด้วย แต่ในส่วนที่ Intermediaries สนใจจะอยู่เฉพาะส่วนที่เป็นหัวข้อ (header) เท่านั้น โดยในส่วนหัวข้อจะมีแอตทริบิวต์ (Attribute) หนึ่งเพื่อใช้ในการระบุว่าใครที่ควรจะทำางกับส่วนหัวนี้ (ผู้ประมวลผลเอกสาร) ซึ่งแอตทริบิวต์นั้นก็คือ actor โดยค่าที่ actor จะต้องใส่ก็คือ URI (Universal Resource Indicator) ของ Intermediary นั้นเอง ดังตัวอย่าง

```
<soap:Header>
  <i:Authentication
    xmlns:i="http://www.i3t.or.th/ws/security"
    soap:actor="urn:I3T:SecurityGateWay"
    soap:mustUnderstand="1">
    <i:username>Acompany</i:username>
    <i:password>SOAPpath</i:password>
  </i:Authentication>
</soap:Header>
```

ดังนั้นเมื่อ Intermediary รับเอกสารSOAPโดยถ้ามีชื่อ attribute actor ตรงกับของตัวเอง Intermediary จะเปลี่ยนเนื้อหาในส่วนหัวที่ได้เพื่อส่งต่อไปยัง Intermediary อื่นหรือไปยังผู้ให้บริการจุดประสงค์หลักที่มีการออกแบบการทำงานแบบนี้ เพื่อลดความซับซ้อนของเอกสารลง นอกจากนี้แล้วยังมียูอาร์แอล (URI) ของแอตทริบิวต์แอคเตอร์ (attribute actor) ชนิดพิเศษอีกหนึ่งค่าคือ “http://schemas.xmlsoap.org/soap/actor/next” เพื่อบอกว่าผู้รับเอกสารSOAPในจุดต่อไปจะเป็นผู้ที่ประมวลผลของเอกสาร ซึ่งกระบวนการนี้จะมีประโยชน์อย่างมากในการส่งเอกสารSOAP เป็นทอดๆ (hop-by-hop)



รูปที่ 2.5 โครงสร้างของเว็บเซอร์วิส